

Final Report

news for u ^-^

SE 4450 — Software Engineering Design II

Capstone Project — 2025/2026

Agile Alligators

Vivek Jariwala | Kaura Sankaran | Sharaf Syed | Scarlet Wu

Faculty Advisor: Dr. Yimin Yang

Co-Faculty Advisor (P. Eng): Dr. Elvis Chen, P.Eng.

Instructor / Sponsor: Dr. Abdelkader Ouda

April 6, 2026

1. Executive Summary

Many people struggle to engage with unfamiliar domains because news articles and commentary assume prior knowledge and rely on industry jargon. The Agile Alligators set out to address this gap by building “news for u,” an AI-powered news platform that aggregates articles from external sources, simplifies them based on a reader’s familiarity level, provides contextual explanations of technical terms, and recommends relevant follow-up content to support progressive learning.

The project was developed over two sprints as part of the SE 4450 capstone course at Western University. Sprint 1 delivered a stable full-stack foundation including user authentication, a news ingestion pipeline, article categorization, and a complete React frontend. Sprint 2 layered AI-driven features on top: article summarization and explanation via Google Gemini, readability scoring via Hugging Face, vector-based embeddings for content similarity, personalized headline feeds, a learning roadmap feature, and a shareable clippings system.

The platform meets its core success criteria: users can register, browse personalized news, receive AI-generated summaries and explanations adapted to their reading level, build learning roadmaps from articles, and save content into shareable clippings. Admin management functionality is fully operational. All 12 system test cases passed on first execution. Areas that remain partially complete include the feedback/learning loop (prototyped but not fully integrated) and per-topic expertise levels (foundation laid via complexity scoring but not exposed in the UI). Notifications and alerts were formally dropped from scope.

2. Introduction & Background

The proliferation of online news has created an information environment that is simultaneously abundant and inaccessible. Domain-specific reporting in fields such as finance, technology, healthcare, and policy routinely uses specialized vocabulary and assumes background knowledge that casual readers may not possess. This creates a barrier to entry: a reader interested in learning about, say, monetary policy may find mainstream coverage incomprehensible without first understanding terms like “quantitative easing” or “yield curve.”

Existing news aggregators (Google News, Apple News, Flipboard) solve the discovery problem but not the comprehension problem. They surface relevant articles but do nothing to adapt the content’s complexity to the reader’s level of understanding. On the other end, educational platforms (Khan Academy, Wikipedia) provide background knowledge but are disconnected from the news cycle. There is a gap for a platform that bridges current events and progressive learning.

Stakeholders

- **End users (readers):** Individuals seeking to engage with unfamiliar topics through simplified, contextualized news content.
- **Faculty advisor and course instructor:** Dr. Yimin Yang (advisor), Dr. Elvis Chen (co-advisor), and Dr. Abdelkader Ouda (instructor/sponsor) provided mentorship, feedback, and grading oversight.
- **Peer teams:** Other SE 4450 teams provided feedback during sprint demos, influencing feature prioritization.

Constraints

- **Budget:** No external funding; AI API usage constrained to free tiers and minimal paid usage (under \$30 for OpenAI/Gemini experimentation).
- **Timeline:** Two sprints across the 2025–2026 academic year, with a hard submission deadline of April 6, 2026.
- **Team distribution:** Two of four members on international exchange during Sprint 2, introducing three time zones.
- **Data access:** Free news APIs often omit article body content, limiting the system’s ability to perform full-text analysis without web scraping (with associated legal considerations).

3. Requirements

3.1 Functional Requirements

The product backlog defined seven epics with 23 user stories. The functional requirements are organized by epic:

- **Epic 1 — User Management:** Account registration with email verification, login with JWT authentication, password reset, account deletion, and admin role management (Stories 1.1–1.4).
- **Epic 2 — Content Aggregation:** Automated fetching from external news APIs, article deduplication, topic-based categorization, and timestamp display (Stories 2.1–2.4).
- **Epic 3 — AI-Driven Adaptation & Explanations:** Article simplification at multiple complexity levels, technical term highlighting, contextual definition popups, and readability scoring (Stories 3.1–3.4).
- **Epic 4 — Personalization & Recommendations:** User topic preference selection, article recommendations based on reading history, feedback on recommendations, and duplicate avoidance (Stories 4.1–4.4).
- **Epic 5 — Search & Discovery:** Keyword search, source filtering, date filtering, and trending topics (Stories 5.1–5.4).
- **Epic 6 — Feedback & Learning Loop:** Quick feedback on simplified articles, flagging unclear definitions, and continuous model improvement via logged feedback (Stories 6.1–6.3). Partially implemented.
- **Epic 7 — Notifications & Alerts:** Topic subscriptions, breaking news alerts, and alert management (Stories 7.1–7.3). Dropped from scope.

3.2 Non-Functional Requirements

- **Confidentiality:** HTTPS for all communication, bcrypt password hashing, role-based access control, minimal PII collection.
- **Integrity:** Server-side input validation, audit logging of sensitive actions, restricted write access by role.
- **Availability:** Regular database backups, reliable hosting environment.
- **Accessibility:** WCAG 2.1 AA design principles including keyboard navigation and high-contrast UI elements.
- **Performance:** Dashboard load within 2 seconds, search results within 2 seconds, learning roadmap generation within 30 seconds.

3.3 Significant Changes from Original Spec

The most significant scope change was the removal of Epic 7 (Notifications & Alerts). During Sprint 2 backlog refinement, the team determined that real-time notifications introduced substantial complexity (push infrastructure, email delivery pipelines, subscription management) without directly contributing to the platform's core value proposition of AI-driven content adaptation. The effort was redirected toward completing the learning roadmap and clippings features, which more directly serve the platform's mission.

The news data source also changed: the team migrated from CurrentsAPI to the Guardian API after discovering that CurrentsAPI did not provide full article body text, which was essential for AI summarization and complexity analysis.

4. Architecture & Design

4.1 System Architecture

The system follows a layered architecture with clear separation of concerns. The architecture diagram (see Appendix A or the project repository) illustrates the following layers:

- **Client Layer:** Web browser accessing the React single-page application.
- **Presentation Layer:** React frontend with JavaScript, using React Context for state management. Responsible for all UI rendering, routing, and user interaction.
- **API Layer:** Express.js server (Node.js) acting as the API gateway. Includes JWT-based authentication middleware and error/validation middleware.
- **Application Services:** Three service groups: (1) News & Content, including the news controller and Guardian service; (2) AI & Embeddings, including the AI service (Google Gemini) and pgvector-based embedding storage; (3) User & Profile, including auth/user management and profile controllers.
- **Data Layer:** PostgreSQL (Yugabyte-compatible) with pgvector extension for storing 3072-dimensional embeddings. Schema managed via migration files in the codebase.
- **External Services:** Guardian News API for article ingestion and Google Generative AI (Gemini) for summaries, explanations, and embeddings.

4.2 Key Design Decisions

Foundation-first, AI-second

The team deliberately built the complete data pipeline and UI shell before introducing any AI components. This decision was motivated by the high uncertainty around LLM integration (cost, latency, output quality). By stabilizing the foundation first, AI features could be layered on without reworking infrastructure.

Gemini over OpenAI for production AI features

After initial exploration of multiple LLM providers, the team selected Google Gemini (gemini-2.5-flash-lite for summarization, gemini-embedding-001 for embeddings) due to favorable cost characteristics, adequate quality for the use case, and straightforward API integration. Hugging Face models (e.g., Qwen/Qwen2.5-7B-Instruct) were used for readability scoring, keeping that workload separate from the generative pipeline.

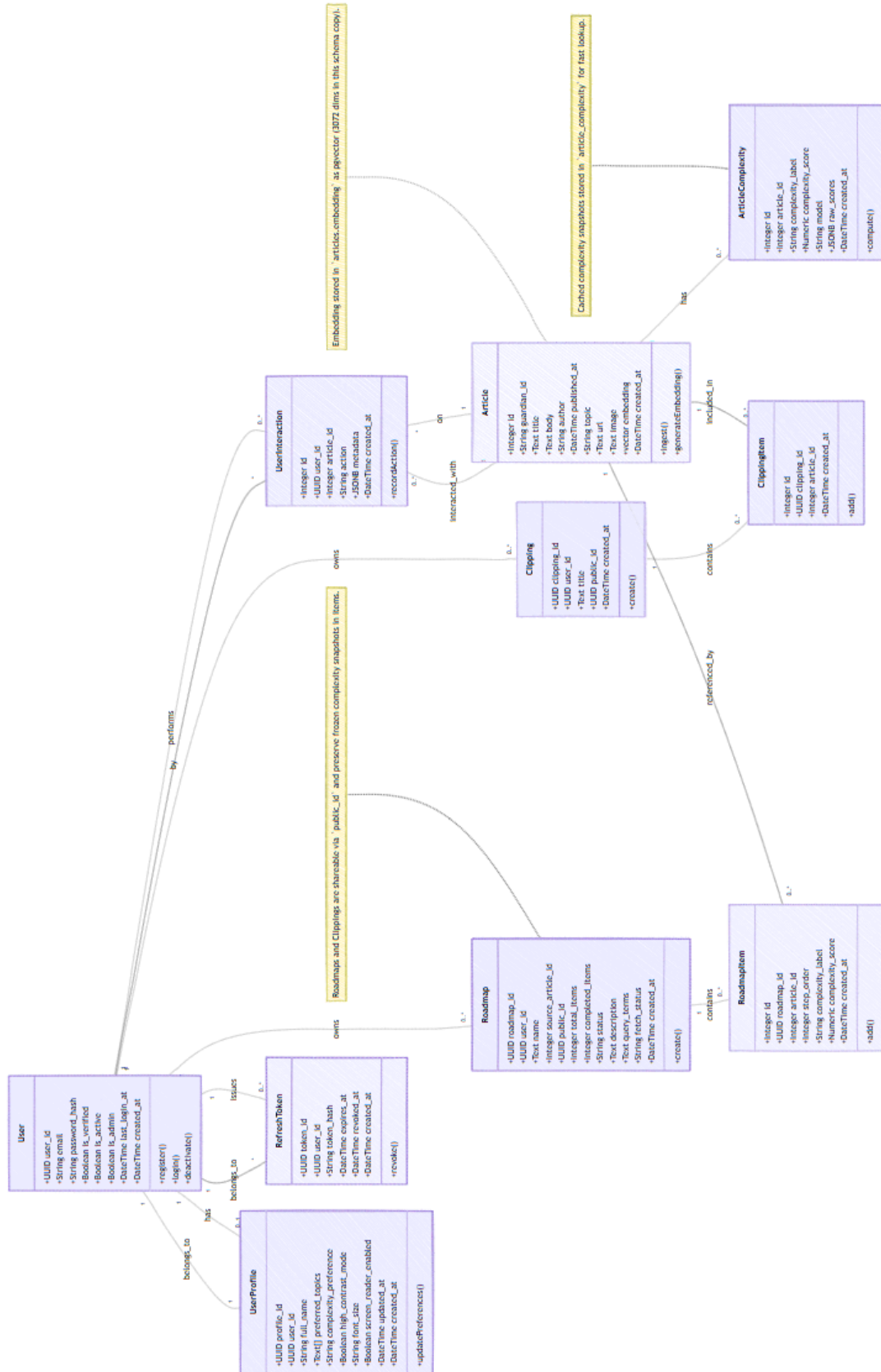
pgvector for embedding storage

Rather than introducing a separate vector database (e.g., Pinecone, Weaviate), the team stored embeddings directly in PostgreSQL using the pgvector extension. This simplified the infrastructure (one database, not two), reduced operational complexity, and was sufficient for the project's scale.

Guardian API migration

CurrentsAPI was replaced with the Guardian API after Sprint 1 because it lacked article body content. The Guardian API provides full article text, richer metadata, and reliable categorization, enabling the full AI summarization and complexity analysis pipeline.

4.3 Data Model



5. Implementation

5.1 Key Components and Modules

Authentication and User Management

User registration and login are handled via `authController.js`, which issues JWT tokens upon successful authentication. Passwords are hashed with `bcrypt`. Email verification is implemented using `nodemailer`, sending a verification link from `news4u.register@gmail.com`. The `profileController.js` manages user preferences (topic interests, reading level) and admin operations (role promotion/demotion, user deletion). All admin endpoints enforce role-based access checks via middleware.

News Ingestion Pipeline

The `guardianService.js` module fetches articles from the Guardian API on a scheduled basis, extracting title, body, source, category, publication date, and thumbnail. Articles are cached to avoid redundant API calls, and deduplication logic prevents identical articles from appearing in user feeds. The `newsController.js` exposes endpoints for fetching headlines (personalized by user topic preferences), article detail retrieval, and keyword search.

AI Services

The `aiService.js` module wraps Google Generative AI (Gemini) and provides three core functions: (1) article summarization using `gemini-2.5-flash-lite`, which rewrites articles at a user-appropriate complexity level; (2) contextual explanation generation, where users can highlight text and receive AI-generated definitions or clarifications; and (3) embedding generation using `gemini-embedding-001`, producing 3072-dimensional vectors stored via `pgvector` for similarity search. Readability scoring uses a Hugging Face model (`Qwen/Qwen2.5-7B-Instruct`) with a custom prompt that labels articles as beginner, intermediate, or advanced.

Learning Roadmaps

When a user clicks “Build Learning Roadmap” on an article, the system generates a multi-step learning path. It retrieves related articles using topic/keyword matching combined with embedding similarity, scores them by complexity, and orders them from easier to more advanced. The result is a sequence of five articles (plus the original) that guides the user from foundational to deeper understanding of the topic. A Gemini-generated summary provides a 2–3 sentence overview of the roadmap.

Clippings System

Users can save articles into named clippings (collections), view their clippings, and share them via public links. The frontend provides a one-click “add to clipping” action from the headline feed. Each clipping page also displays a carousel of recommended related articles based on the clipping’s topic profile.

5.2 Third-Party Libraries and APIs

Library / Service	Purpose	Notes
React	Frontend framework	Create React App scaffolding with React Context for state management.

Express.js	Backend API	Node.js web framework for RESTful API endpoints.
PostgreSQL / Yugabyte	Database	Relational storage with pgvector extension for embeddings.
Google Generative AI (Gemini)	AI summarization, explanations, embeddings	gemini-2.5-flash-lite for text generation; gemini-embedding-001 for 3072-dim vectors.
Hugging Face (Qwen2.5-7B)	Readability scoring	Classifies article complexity as beginner/intermediate/advanced.
Guardian News API	Article ingestion	Replaced CurrentsAPI in Sprint 2 for full article body access.
bcrypt / JWT / nodemailer	Auth infrastructure	Password hashing, session tokens, email verification.

5.3 Challenges and Resolutions

- **LLM integration delay:** The team spent Sprint 1 without converging on an AI provider. Resolved by committing to Google Gemini early in Sprint 2 on advisor recommendation, with a budget-conscious approach.
- **Missing article body content:** CurrentsAPI returned only metadata and headlines, not full text. Resolved by migrating to the Guardian API, which provides complete article bodies.
- **Embedding storage at scale:** Initially considered a separate vector database, but the added operational complexity was not justified at the project's scale. Resolved by using pgvector within the existing PostgreSQL instance.
- **Distributed team coordination:** Two members on exchange introduced three time zones. Resolved (partially) by establishing fixed weekly standups and using asynchronous pull-request-based workflows.

6. Verification & Testing

6.1 Testing Strategy

The team employed a system-level functional testing approach, supplemented by manual regression testing after bug fixes or feature updates. Test cases were designed to cover the core end-to-end user journeys: account lifecycle, article interaction, profile management, admin operations, and the learning roadmap feature. Testing tools included manual UI walkthroughs and Postman for backend API endpoint verification.

The team had initially planned to build a pytest-based automated test suite, but time constraints during Sprint 2 limited its reliability. As a result, the primary testing method was structured manual execution against the defined test cases.

6.2 Test Coverage Summary

Twelve system test cases were defined and executed, covering the following areas:

TC ID	Title	Date	Result	Notes
TC-01	User creates account	Mar 15	Pass	Verification email triggered
TC-02	User authenticates and accesses home page	Mar 15	Pass	Redirected to dashboard
TC-03	Search returns relevant news results	Mar 15	Pass	10 relevant articles returned
TC-04	Saved articles appear in clipping list	Mar 15	Pass	Immediate UI update
TC-05	Saved articles removed from clipping list	Mar 15	Pass	Confirmed after refresh
TC-06	Topics updates persist after saving	Mar 15	Pass	Persisted on refresh
TC-07	Alerts updates persist after saving	Mar 15	Pass	Persisted after logout/login
TC-08	User admin status persists after login	Mar 15	Pass	Non-admin blocked from /admin
TC-09	Admins can change other users' admin status	Mar 15	Pass	Confirmed cross-account
TC-10	Admins can delete users	Mar 15	Pass	Login returns not-found
TC-11	Verification email works [MANUAL]	Mar 15	Pass	Email received in 2 min
TC-12	User can build learning roadmap	Mar 15	Pass	5 articles, ~18s load

All 12 test cases passed on first execution with no defects raised. No re-runs were required.

6.3 Known Defects and Limitations

- **No automated regression suite:** The planned pytest suite was not completed. All testing was manual, which limits repeatability and confidence during rapid integration.
- **Performance testing not conducted:** Load testing, API response time benchmarking, and cache repopulation timing were out of scope for this testing phase.
- **Security testing limited:** Beyond basic role-based access validation, no penetration testing or injection testing was performed.
- **Roadmap generation latency:** The learning roadmap takes approximately 18–20 seconds to generate, which may impact user experience. Optimization (caching, pre-computation) was not addressed.

7. Results & Discussion

The platform was evaluated against the original project objectives and Sprint 1 KPIs:

Objective	Target	Outcome	Assessment
User management	85% of CRUD	Achieved	Registration, login, email verification, profile management, account deletion, and admin role management all functional.
Content aggregation	50%	Exceeded	Full article ingestion via Guardian API with caching, deduplication, categorization, and personalized feeds.
AI explanations & personalization	30%	Exceeded	Gemini-powered summaries, explanations, complexity scoring, embeddings, and learning roadmaps delivered.
Notifications	40%	Dropped	Formally removed from scope to focus on core AI-driven value.
Clean UI	60%	Achieved	Full React frontend with routing, responsive components, carousel layouts, and article card actions.

Peer feedback from Sprint 1 demos was positive regarding UX quality and layout. Recurring suggestions included adding visible recommendation indicators (e.g., “Recommended for you” labels) and richer article metadata in previews. The former was partially addressed through the recommendation and clippings infrastructure; the latter remains a future enhancement.

The AI features represent the platform’s primary differentiator. The combination of Gemini-powered summarization, Hugging Face readability scoring, and the learning roadmap feature creates a user journey that no mainstream news aggregator currently provides: read an article, get an AI-generated summary at your level, then follow a curated learning path from simpler to more complex coverage of the same topic.

8. Conclusions & Future Work

8.1 Summary of Accomplishments

The Agile Alligators delivered a functional, AI-powered news platform that addresses the core problem statement: making news more accessible to readers unfamiliar with a topic. The system aggregates articles from the Guardian API, generates AI summaries and contextual explanations via Google Gemini, scores article complexity using Hugging Face models, builds learning roadmaps ordered by difficulty, and provides a clippings system for saving and sharing content. User management, authentication, and admin features are fully operational. All 12 system test cases passed.

8.2 Honest Reflection on Shortcomings

The feedback and learning loop (Epic 6) was only partially completed. While the concept and prototype were developed, full integration—where user feedback actively influences AI model behavior—was not achieved. This limits the system’s ability to improve over time based on real user interactions.

Notifications (Epic 7) were dropped entirely. While this was a deliberate scope decision, it means the platform lacks proactive engagement mechanisms that would increase user retention.

Testing remained primarily manual. The absence of an automated regression suite means that future changes carry higher risk of introducing undetected regressions.

The LLM integration strategy was finalized too late, consuming most of Sprint 2’s capacity and leaving less room for polish and secondary features.

8.3 Recommended Future Enhancements

1. **Per-topic expertise levels:** Allow users to set different familiarity levels per topic (e.g., expert in technology, beginner in economics) rather than a single global reading level. The complexity scoring infrastructure already supports this.
2. **Closed-loop feedback system:** Complete the feedback pipeline so that user ratings on simplified articles and flagged definitions are logged, aggregated, and used to fine-tune prompts or retrain models on a scheduled basis.
3. **Recommendation transparency:** Add visible indicators explaining why articles are recommended (e.g., “Based on your interest in climate policy” or a match-score badge), improving user trust and engagement.
4. **Multi-source bias comparison:** Enable users to compare how different outlets cover the same topic, supporting more balanced and critical news consumption.
5. **Automated testing and CI/CD:** Build out the pytest suite (or equivalent) and integrate it into a GitHub Actions CI pipeline for automated regression testing on every pull request.

9. References

The Guardian Open Platform API. <https://open-platform.theguardian.com/>

Authentication and Security:

- JSON Web Tokens (JWT) Introduction. <https://jwt.io/introduction>
- bcrypt npm package. <https://www.npmjs.com/package/bcrypt>
- Nodemailer Documentation. <https://nodemailer.com/>

Database:

- PostgreSQL Documentation. <https://www.postgresql.org/docs/>
- YugabyteDB Documentation. <https://docs.yugabyte.com/>
- node-postgres (pg) Documentation. <https://node-postgres.com/>

AI/ML Specific Models:

- Qwen2.5-7B-Instruct Model Card, Hugging Face. <https://huggingface.co/Qwen/Qwen2.5-7B-Instruct>
- Google Generative AI (Gemini) API Documentation. <https://ai.google.dev/>
- pgvector: Open-source vector similarity search for PostgreSQL. <https://github.com/pgvector/pgvector>

Accessibility:

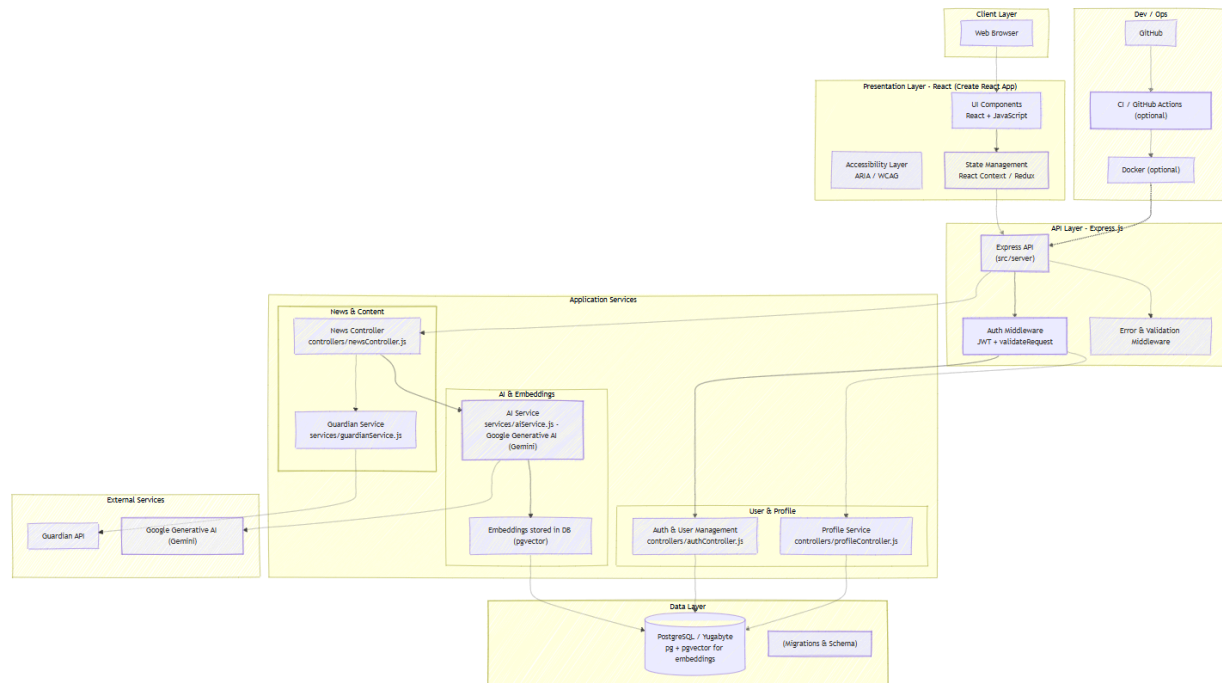
- WAI-ARIA Authoring Practices, W3C. <https://www.w3.org/WAI/ARIA/apg/>
- Web Content Accessibility Guidelines (WCAG) 2.1, W3C Recommendation. <https://www.w3.org/TR/WCAG21/>

Development Tools:

- Docker Documentation. <https://docs.docker.com/>
- GitHub Actions Documentation. <https://docs.github.com/en/actions>
- Postman API Platform. <https://www.postman.com/>

10. Appendices

Appendix A — Architecture Diagram



Appendix B — Full System Test Cases

The complete system test case definitions (TC-01 through TC-12) with pre-conditions, step-by-step instructions, and expected results are documented in the System Test Cases file (docs/System Testing/System_Test_Cases.docx). The test execution log with results is in Test_Execution_Log.docx.

Appendix C — Product Backlog

The full product backlog with all 7 epics and 23 user stories, including priority, story points, acceptance criteria, and sprint targets, is available in the repository as Product_Backlog.csv.